Quiz

Which recurrence describes the running time of Mergesort for $n \geq 1$, assuming n is a power of two?

- a. $T(n) = 2T(n/2) + \Theta(n^2)$
- b. $T(n) = 2T(n/2) + \Theta(n)$ (Zwei Hälften rekursiv lösen, Merge)
- c. $T(n) = 2T(n-1) + \Theta(n^2)$
- d. $T(n) = 2T(n/2) + \Theta(1)$

Consider the Quicksort algorithm. Given the array $[7, 2, 9, 4, 6]$ (p), the last element is chosen as the pivot. At which position in the array is the pivot after completely partitioning all elements? Positions are numbered starting at 1.

Aufteilen → [4, 2, 6, 7, 9] (1 2 3 4 5)

≤ 6 | ≥ 6

Answer: 3

Quicksort can partition an array *in place*, using only $O(1)$ additional storage.

(for variables i, j, p)

- True
- False

In the lecture you have seen that a complete binary tree can be stored as an array, where nodes are stored level by level, from left to right, starting at index 1 of the array.
Which of the following array representations of a complete binary tree corresponds to a valid Max-Heap? Choose exactly one answer.

- a. $[7, 9, 8, 3, 5, 6]$ 
- b. $[9, 7, 8, 3, 5, 6]$  ✔
- c. $[9, 7, 6, 3, 5, 8]$ 
- d. $[9, 7, 8, 3, 10, 6]$ 

Wie umformen:

[1, 2, 3, 4, 5, 6] → 

Kinder von i bei 2i und 2i+1

Why must the decision tree of a correct comparison-based sorting algorithm for n distinct elements contain at least $n!$ leaves?

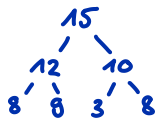
- a. Because every path in the decision tree contains exactly $n!$ comparisons.
- b. Because each comparison has n possible outcomes. no, it has 2 possible outcomes
- [x] c. Because $n!$ different orderings of the input elements must be distinguished. = permutations
- d. Because every element must be compared with every other element. this would only be $\frac{n(n-1)}{2}$ comparisons

A Max-Heap contains the key 15 at its root. Where would you continue searching for the key 8?

- a. In the left subtree.
- b. In the right subtree.
- [x] c. We cannot know which of the two subtrees might contain the key.

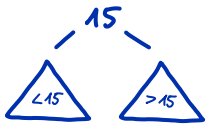
Max-Heap ≠ Binary Search tree

e.g.

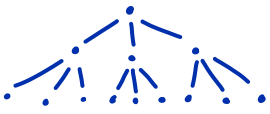


A binary search tree contains the key 15 at its root. Where would you continue searching for the key 8?

- a. In neither subtree, because we already know that the key is not contained in the binary search tree.
- b. In the right subtree.
- [x] c. In the left subtree. because



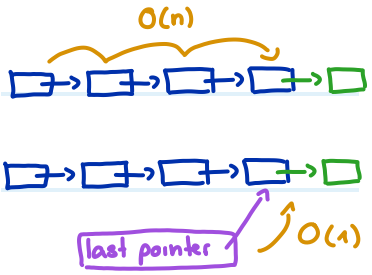
A 2-3 tree of height h has at most 3^h leaves.



h=0, 3^0=1 Knoten
h=1, 3^1=3 Knoten
h=2, 3^2=9 Knoten

- [x] True
- False

A singly linked list L already stores a pointer to its first element. Assume that an additional pointer to its last element is now stored. Which operation improves from $\Theta(n)$ to $\Theta(1)$ in the worst case, where n denotes the length of the list?



- ○ a. Searching for a given key value.
- ☒ b. Appending a new key k at the end using $\texttt{insert}(k, L)$.
- ○ c. Obtaining the memory address of the i-th element using $\texttt{get}(i, L)$, where $i \in \{1, \ldots, n\}$ is arbitrary.
- ○ d. Deleting an arbitrary node whose memory address is known.

Consider the abstract datatype (ADT) *List*. The memory address of the node to be deleted is given. The address of its predecessor is **not** provided separately. Which statement describes the worst-case running time for the deletion of the node in a list with n elements?

- ☒ a. Singly linked lists require $\Theta(n)$, while doubly linked lists require $\Theta(1)$.
  ↳ we need to redirect the pointer of the predecessor
- ○ b. Both singly and doubly linked lists require $\Theta(n)$.
- ○ c. Both singly and doubly linked lists require $\Theta(1)$.
- ○ d. Singly linked lists require $\Theta(1)$, while doubly linked lists require $\Theta(n)$.

What is the running time of the following program?

FIBONACCI(n)

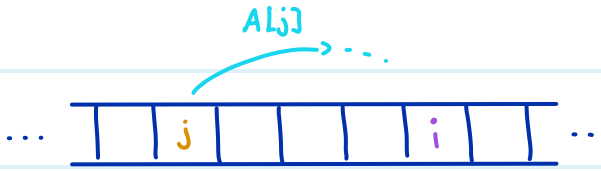
```
1 F[1..n] ← new array
2 F[1] ← 1
3 F[2] ← 1
4 for i ← 3, ..., n do
5     F[i] ← F[i − 1] + F[i − 2]
6 return F[n]
```

- ○ a. $\Theta(1)$
- ☒ b. $\Theta(n)$ one loop from i=3...n
- ○ c. At least exponential
- ○ d. None of the other options

Consider the jump game from the lecture with given input array $A[1..n]$. Let $S[i]$ denote the minimum number of jumps needed to reach reach i. Which of the following expressions is the correct recursion for $S[i]$?

- [x] a. $S[i] = \min\{S[j] + 1 \mid 1 \leq j < i \text{ and } j + A[j] \geq i\}$
- [ ] b. $S[i] = \min\{A[j] + 1 \mid 1 \leq j < i \text{ and } j + A[j] \leq i\}$
- [ ] c. $S[i] = \min\{A[j] + 1 \mid 1 \leq j < i \text{ and } j + A[j] \geq i\}$
- [ ] d. $S[i] = \min\{S[j] + 1 \mid 1 \leq j < i \text{ and } j + A[j] \leq i\}$

let j be the index from where we jump to index i



S[j] = min. jumps to reach j

$S[i] = \min \{ S[j] + 1 \mid 1 \leq j < i \text{ and } j + A[j] \geq i \}$

S[j] + 1: one more jump

1 ≤ j < i: j is a predecessor

j + A[j] ≥ i: starting at j, our jump is long enough to reach i

For the edit distance $ED(n, m)$ between two given strings $A[1..n]$ and $B[1..m]$, assume that after the optimal sequence of edits, the entry $A[n]$ is not deleted and ends up in a position in $B[1..m-1]$. What is then necessarily true?

=> B[m] must be added to the end of A

- [ ] a. $ED(n, m) = \begin{cases} 0 & \text{if } A[n] = B[m] \\ 1 & \text{if } A[n] \neq B[m] \end{cases}$
- [ ] b. $ED(n, m) = 1 + ED(n - 1, m)$
- [ ] c. $ED(n, m) = 1 + ED(n - 1, m - 1)$
- [x] d. $ED(n, m) = 1 + ED(n, m - 1)$

corresponds to adding B[m] to A
<=> deleting B[m] from B

$a_1, a_2, \ldots, a_{n-2}, a_{n-1}, \boxed{a_n}$ kept

$b_1, b_2, \ldots, b_{m-2}, b_{m-1}, \boxed{b_m}$ deleted from B
<=> added to A

Discussion of Exercise Sheet 5

Exercise 5.3 *Counting function calls in recursive functions* **(2 bonus points)**.

For each of the following functions g, h, and k, compute the number of calls to f as a function of n, including calls made during recursive calls. Give **both** the exact number of calls as well as a maximally simplified asymptotic bound in Θ notation. For the exact number, you may use summation signs in your expression. You can assume that n is a power of two.

Algorithm 2

```
function h(n)
    i ← 1
    while i < n do          <=> i ≤ n-1
        f()
        i ← i + 1
    k(n)
    k(n)
function k(n)
    if n = 1 then
        return
    i ← 2
    while i < n do          i ≤ ⌈log log n⌉   <=>   i ← i+1
        f()
        i ← i²
    h(n/2)
```

(1) $H(n) = \left(\sum_{i=1}^{n-1} 1\right) + 2 \cdot K(n) = n - 1 + 2 \cdot K(n)$ für $n \geq 2$

und $H(1) = 0$

(2) $K(n) = \left(\sum_{i=1}^{\lceil \log \log n \rceil} 1\right) + H\left(\frac{n}{2}\right) = \lceil \log \log n \rceil + H\left(\frac{n}{2}\right)$ für $n \geq 2$

und $K(1) = 0$

Überlegung:

Iteration	Wert von i (am Ende der Iteration)
0	$2 = 2^1$
1	2^2 (hoch 2)
2	$(2^2)^2 = 2^4$ (hoch 2)
3	$(2^4)^2 = 2^8$ (hoch 2)
4	$(2^8)^2 = 2^{16}$ (hoch 2)
⋮	⋮
k	2^{2^k}

Sei $n = 2^{2^k} \Leftrightarrow \log n = 2^k \Leftrightarrow \log \log n = k$

$\Rightarrow$ Anz Iterationen $= k = \log \log n$

Frage: Auf- oder abrunden?

Bsp. Sei $n = 8$, $\underbrace{2^{2^1}}_{k=1} < 8 < \underbrace{2^{2^2}}_{k=2}$

Iteration	Wert von i am Anfang der Iteration	call?
1	2	✓
2	4	✓
3	16	✗

$\log \log 8 = \log 3 \approx 1.6 \rightarrow$ soll zu 2 calls aufgerundet werden

(2) in (1) einsetzen.

$$H(n) = n-1+2\Big(\overbrace{\lceil \log\log n \rceil}^{K(n)} + H\left(\tfrac{n}{2}\right)\Big)$$

$$= 2H\left(\tfrac{n}{2}\right) + n - 1 + 2\lceil \log\log n \rceil$$

$$= 2\cdot\left(2H\left(\tfrac{n}{4}\right) + \tfrac{n}{2} - 1 + 2\lceil \log\log\left(\tfrac{n}{2}\right)\rceil\right) + n - 1 + 2\lceil \log\log n \rceil$$

$$= 2\left(2\cdot\left(2H\left(\tfrac{n}{8}\right) + \tfrac{n}{4} - 1 + 2\lceil \log\log\left(\tfrac{n}{4}\right)\rceil\right) + \tfrac{n}{2} - 1 + 2\lceil \log\log\left(\tfrac{n}{2}\right)\rceil\right) + n - 1 + 2\lceil \log\log n \rceil$$

Teleskopieren

$$\vdots$$

$$= \left(n - 1 + 2\lceil \log\log n \rceil\right) + 2\left(\tfrac{n}{2} - 1 + 2\lceil \log\log\left(\tfrac{n}{2}\right)\rceil\right) + 2^2\left(\tfrac{n}{4} - 1 + 2\lceil \log\log\left(\tfrac{n}{4}\right)\rceil\right) + \ldots + 2^{(\log n)-1}\cdot\left(2 - 1 + 2\lceil \log\log 2 \rceil\right)$$

$$= \sum_{j=1}^{\log n} \frac{n}{2^j}\cdot\left(2^j - 1 + 2\lceil \log\log(2^j)\rceil\right) = \sum_{j=1}^{\log n} \frac{n}{2^j}\cdot\left(2^j - 1 + 2\lceil \log j \rceil\right) \qquad (3)$$

$\log(2^j) = j$

Es gilt: $\frac{2^j}{2} \leq \left(2^j - 1 + 2\lceil \log j \rceil\right) \leq 3\,2^j$ (durch Einsetzen überprüfen)

$\Rightarrow \frac{n}{2^j}\cdot\frac{2^j}{2} \leq \frac{n}{2^j}\left(2^j - 1 + 2\lceil \log j \rceil\right) \leq \frac{n}{2^j}\cdot 3\,2^j$ (mit $\frac{n}{2^j}$ multiplizieren)

$\Rightarrow \frac{n}{2} \leq \frac{n}{2^j}\left(2^j - 1 + 2\lceil \log j \rceil\right) \leq 3n$

$\Rightarrow$ alle $\log(n)$ Terme der Summe sind in $\Theta(n)$

$\Rightarrow H(n) = \Theta(n \log n)$

(3) in (2) einsetzen.

$$K(n) = \lceil \log\log n \rceil + H\left(\tfrac{n}{2}\right)$$

$$= \lceil \log\log n \rceil + \sum_{j=1}^{\log \frac{n}{2}} \frac{\frac{n}{2}}{2^j}\cdot\left(2^j - 1 + 2\lceil \log j \rceil\right)$$

Analoge Abschätzung hier nochmal für $K(n)$ machen.. $\Rightarrow K(n) = \Theta(n \log n)$

ADT Wörterbuch

Gewünschte Operationen:

1) `search`(x, W): Entscheide, ob der Schlüssel x in W vorkommt, und gib in diesem Fall die Position im Speicher zurück.

2) `insert`(x, W): Füge den Schlüssel x in W ein, falls er noch nicht darin vorkommt.

3) `delete`(x, W): Finde und lösche den Schlüssel x aus W, falls er darin vorkommt.

=> wir müssen immer zuerst suchen, ob x schon enthalten ist

Mögliche Implementierungen.

	search	insert	delete
unsortiertes Array	$O(n)$	$O(n)$	$O(n)$
sortiertes Array	$O(\log n)$ *	$O(n)$ *	$O(n)$
Doppelt verk. Liste	$O(n)$	$O(n)$	$O(n)$
Binärer Suchbaum	$O(h)$	$O(h)$	$O(h)$
2-3-Baum	$O(\log n)$	$O(\log n)$	$O(\log n)$

mit BinarySearch

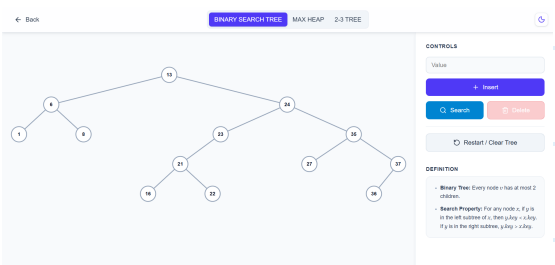
beim Einfügen am Anfang müssen O(n) Elemente nach hinten gerückt werden

} heute

Binärer Suchbaum:

- Jeder Knoten hat maximal 2 Kinder
- Suchbaum-Bedingung:

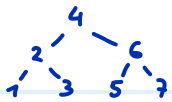
alle Schlüssel im linken Teilbaum < Schlüssel des Knotens < alle Schlüssel im rechten Teilbaum



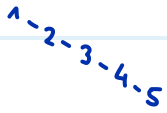
- Alle Operationen in $O(h)$, wobei h die Höhe des Baumes ist

  (die Operation Löschen ist nicht prüfungsrelevant)

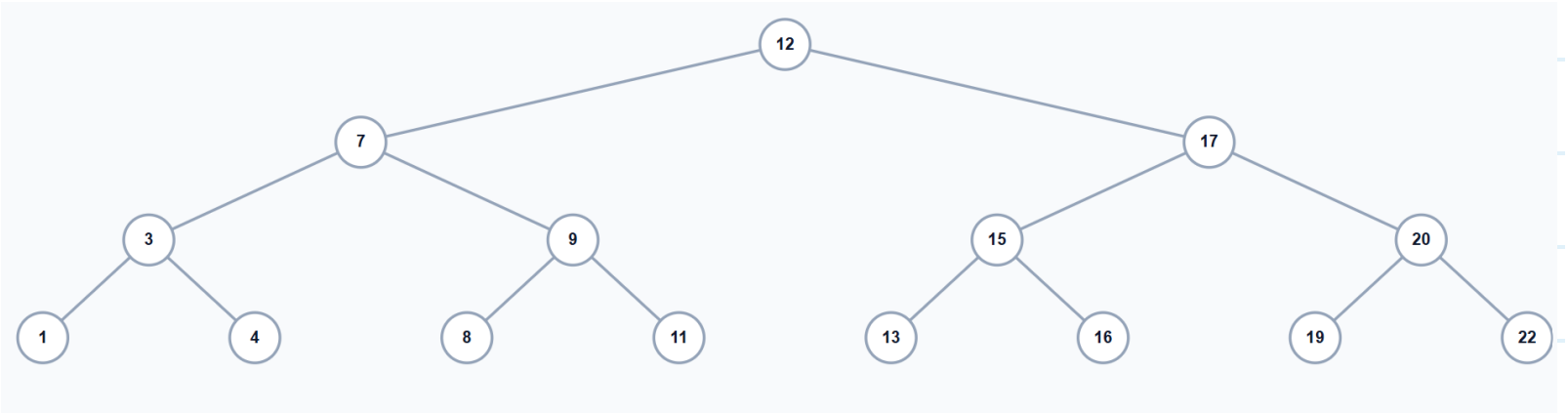
- Gut falls $h = \Theta(\log n)$ ist (balanciert)



- Schlecht falls $h = \Theta(n)$ ist (unbalanciert)



Verbindung zwischen binary search und binary search tree:



als Liste:

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
1	3	4	7	8	9	11	12	13	15	16	17	19	20	22

search(8):

							m=8							r=15
l=1														
1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
1	3	4	7	8	9	11	12	13	15	16	17	19	20	22

l=1			m=4			r=7								
1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
1	3	4	7	8	9	11	12	13	15	16	17	19	20	22

				l=5	m=6	r=7								
1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
1	3	4	7	8	9	11	12	13	15	16	17	19	20	22

				l=r=5										
1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
1	3	4	7	8	9	11	12	13	15	16	17	19	20	22

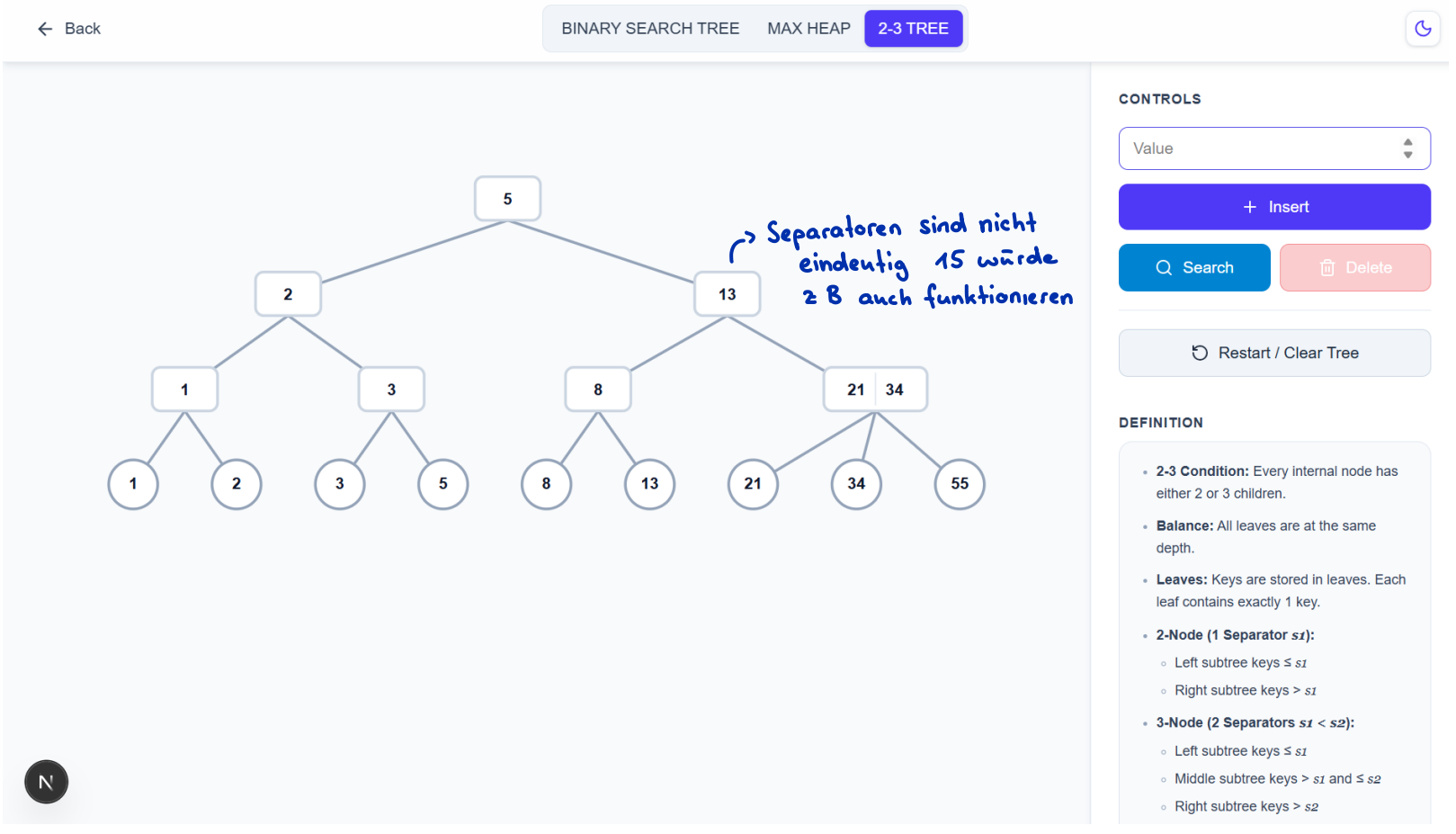
Verbesserung: 2-3-Bäume

(Achtung: unterschiedliche Namen in anderen Quellen)

Ein 2-3-Baum für $n \geq 2$ Schlüssel erfüllt die folgenden Bedingungen:

- Jeder innere Knoten hat entweder 2 oder 3 Kinder. ("2-3-Bedingung")
  Knoten die keine Blätter sind
- Alle Blätter befinden sich auf derselben Ebene (Tiefe). => perfekt balanciert, $h = \Theta(\log n)$
- Die Schlüssel sind in den Blättern gespeichert. Jedes Blatt enthält genau einen Schlüssel.
- Ein Knoten mit 2 Kindern enthält einen Separator s_1. Alle Schlüssel im linken Teilbaum sind $\leq s_1$, und alle im rechten sind $> s_1$ ("Separatorbedingung bei zwei Kindern").
- Ein Knoten mit 3 Kindern enthält zwei Separatoren s_1 und s_2 (mit $s_1 < s_2$). Für die Schlüssel in den drei Teilbäumen gilt ("Separatorbedingung bei drei Kindern"):
  - linker Teilbaum: alle Schlüssel $\leq s_1$,
  - mittlerer Teilbaum: alle Schlüssel $> s_1$ und $\leq s_2$,
  - rechter Teilbaum: alle Schlüssel $> s_2$.

Merkhilfe "Bei Gleicheit nach links"



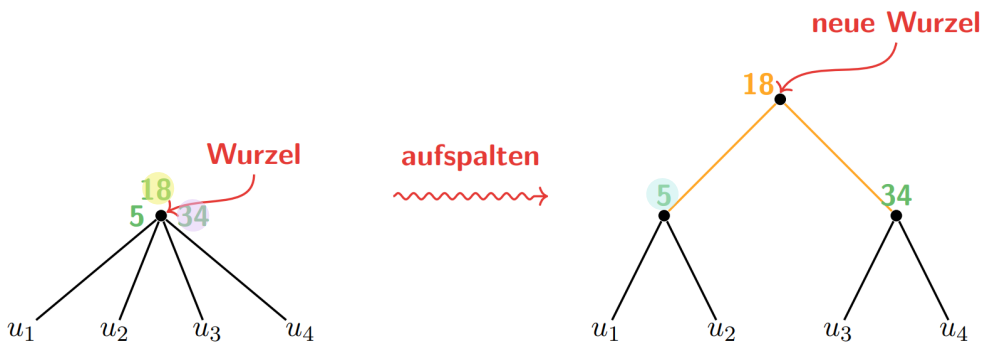
search(x): Bei der Wurzel beginnen und den Separatoren entlang folgen

insert(x): 1) Suche richtigen Platz für x

2) Erstelle ein neues Blatt mit Schlüssel x, und füge beim Elternknoten einen passenden Separator hinzu.

3) Falls der Elternknoten nun 4 Kinder hat, teile ihn in zwei Knoten auf die den kleinsten und grössten Separator übernehmen, und schiebe den mittleren Separator eine Ebene nach oben

Falls die 2-3-Bedingung dadurch ein Level weiter oben verletzt wurde, repariere rekursiv.



delete(x): 1) Suche x

2) Lösche x und den dazugehörenden Separator

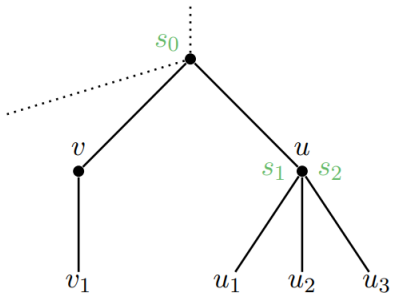
3.) Falls 2-3-Bedingung verletzt: (= nur noch 1 Kind)

i) Falls ein Geschwisterknoten 3 Kinder hat, dann adoptieren wir eines und aktualisieren die dazugehörenden Separatoren

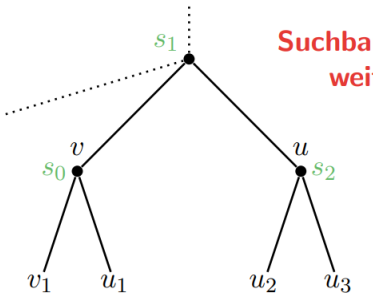
ii) Sonst verschmelze den Knoten mit einem Geschwisterknoten (der 2 Kinder hat)

Falls die 2-3-Bedingung dadurch ein Level weiter oben verletzt wurde, repariere rekursiv.

Bsp. Fall (i)

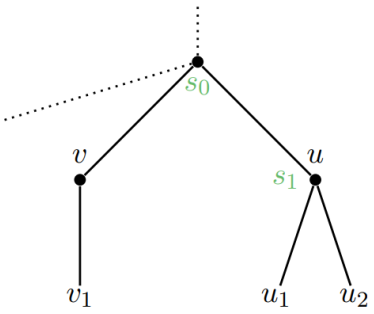


Adoption

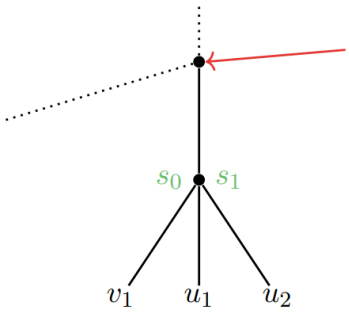


Suchbaumbedingung weiter erfüllt

Bsp. Fall (ii)

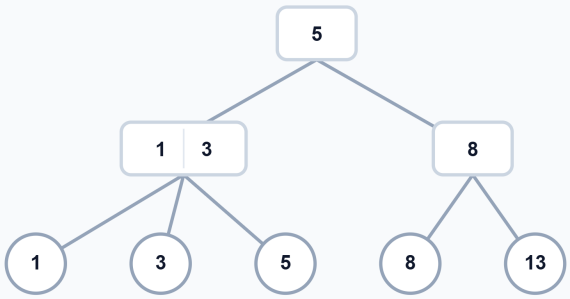


Verschmelzen

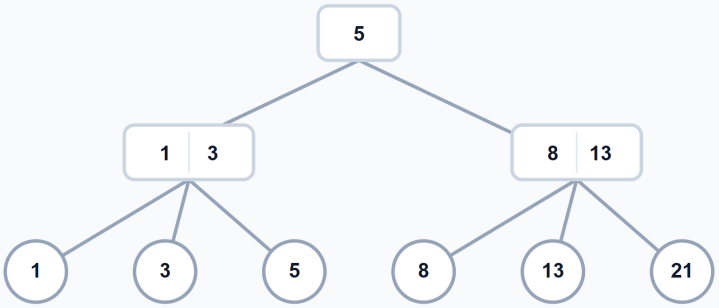


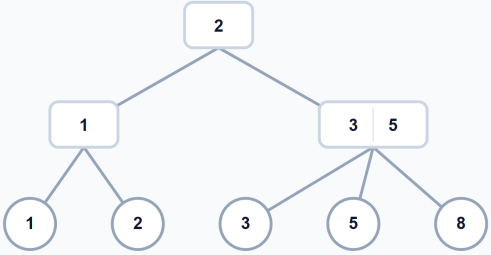
verliert ein Kind

Aufgaben:

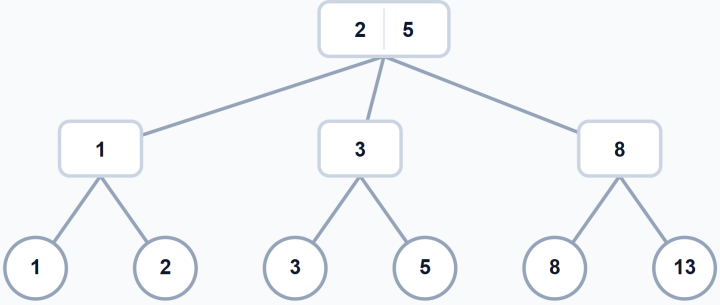


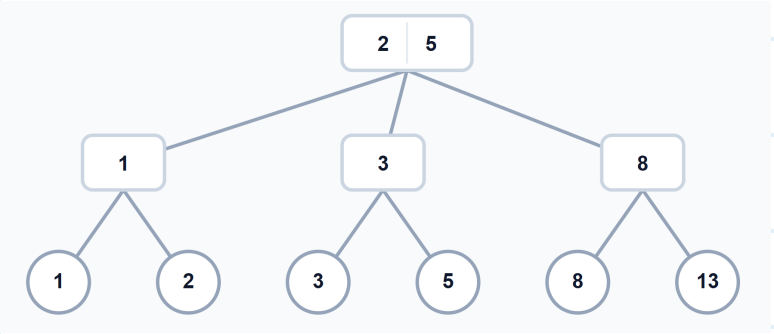
insert(21)



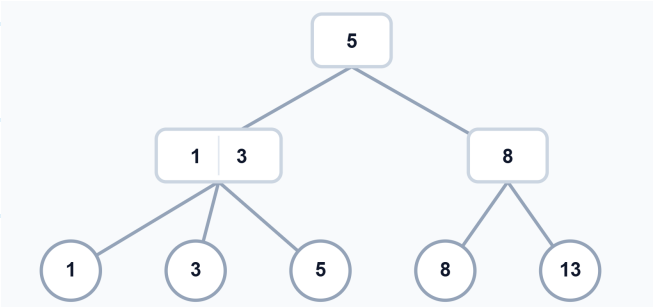


insert(13)

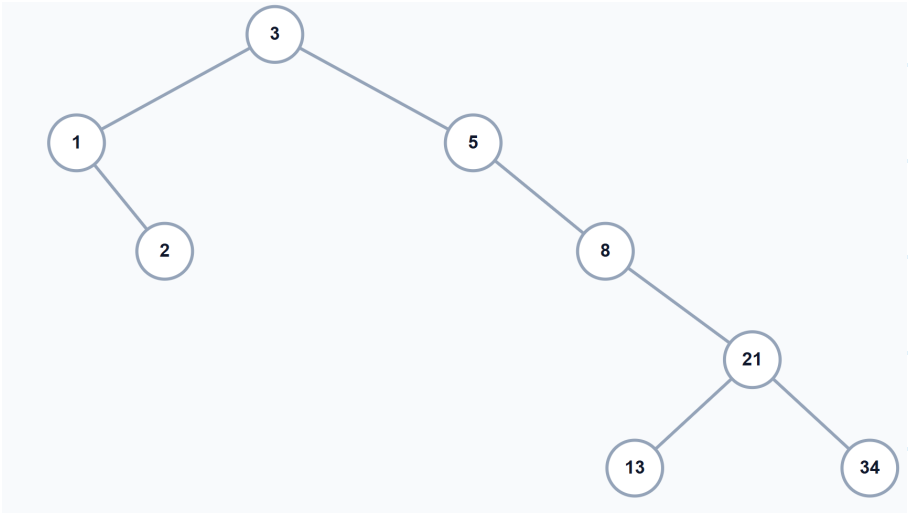




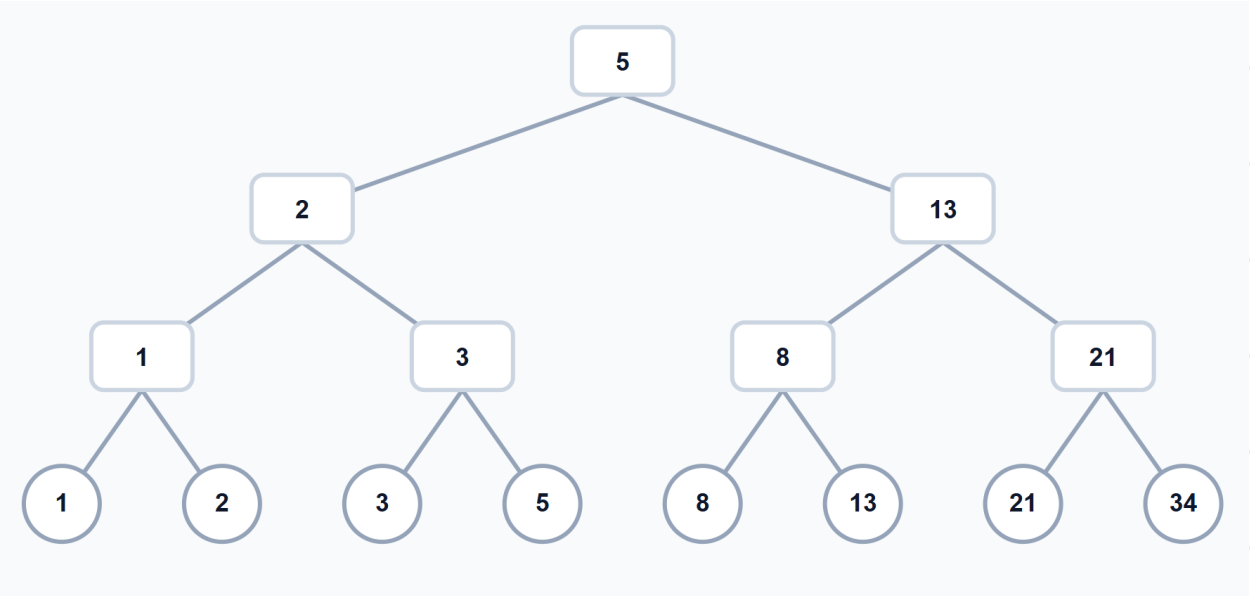
delete(2)



Füge die Schlüssel 3, 1, 2, 5, 8, 21, 13, 34 in dieser Reihenfolge in einen leeren binären Suchbaum:



Challenge: Füge die Schlüssel 3, 1, 2, 5, 8, 21, 13, 34 in dieser Reihenfolge in einen leeren 2-3-Baum:



Dynamische Programmierung

Herangehensweise

1.) Dimension der DP-Tabelle definieren

2) Teilproblem definieren, d.h. die Bedeutung eines DP-Entries festlegen

3.) Base Cases und Rekursion bestimmen

4.) Berechnungsreihenfolge

5) Lösung aus der DP-Tabelle auslesen

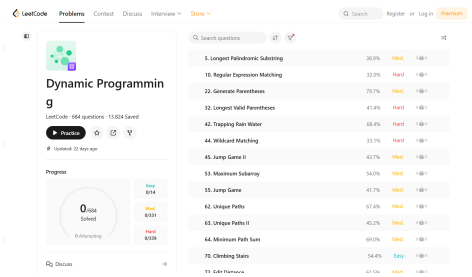
6.) Laufzeit analysieren

Zwei Varianten zum Programmieren:

- Bottom-up (= iterativ) — ist am Anfang einfacher
- Top-down (= rekursiv = mit memoization) — besonders geeignet wenn man sich nicht um die Berechnungsreihenfolge kümmern möchte

Wie lernen?

- möglichst viele Aufgaben lösen
- mit einfachen Aufgaben beginnen und sie Bottom-up und Top-down programmieren
- Zusatzaufgaben auf leetcode.com sind sehr hilfreich



Fibonacci

Dimension: $F[1 \dots n]$ — alternative: $F[0 \dots n]$

Teilproblem: $F[i]$:= i-te Fibonacci Zahl

Base Case: $F[1] = 1$ und $F[2] = 1$ — alternative: $F[0] = 0$ und $F[1] = 1$

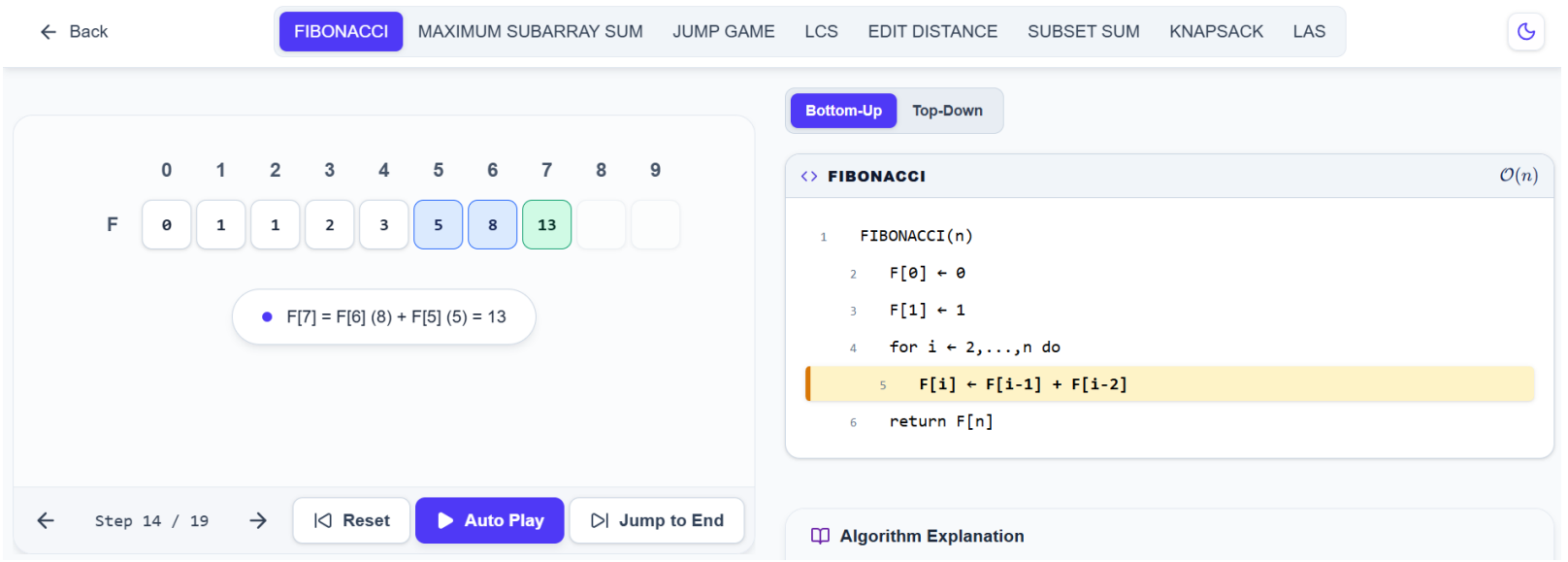
2 Base Cases weil sich die Rekursion auf 2 vorherige Entries bezieht

Rekursion: $F[i] = F[i-1] + F[i-2]$ für $3 \leq i \leq n$

Berechnungsreihenfolge: for $i = 3 \dots n$: berechne $F[i]$

Lösung: $F[n]$

Laufzeit: $O(n)$



MSS

Dimension: $R[1..n]$

Teilproblem: $R[i]$:= maximale Summe eines Teilarrays von A, das bei Index i endet

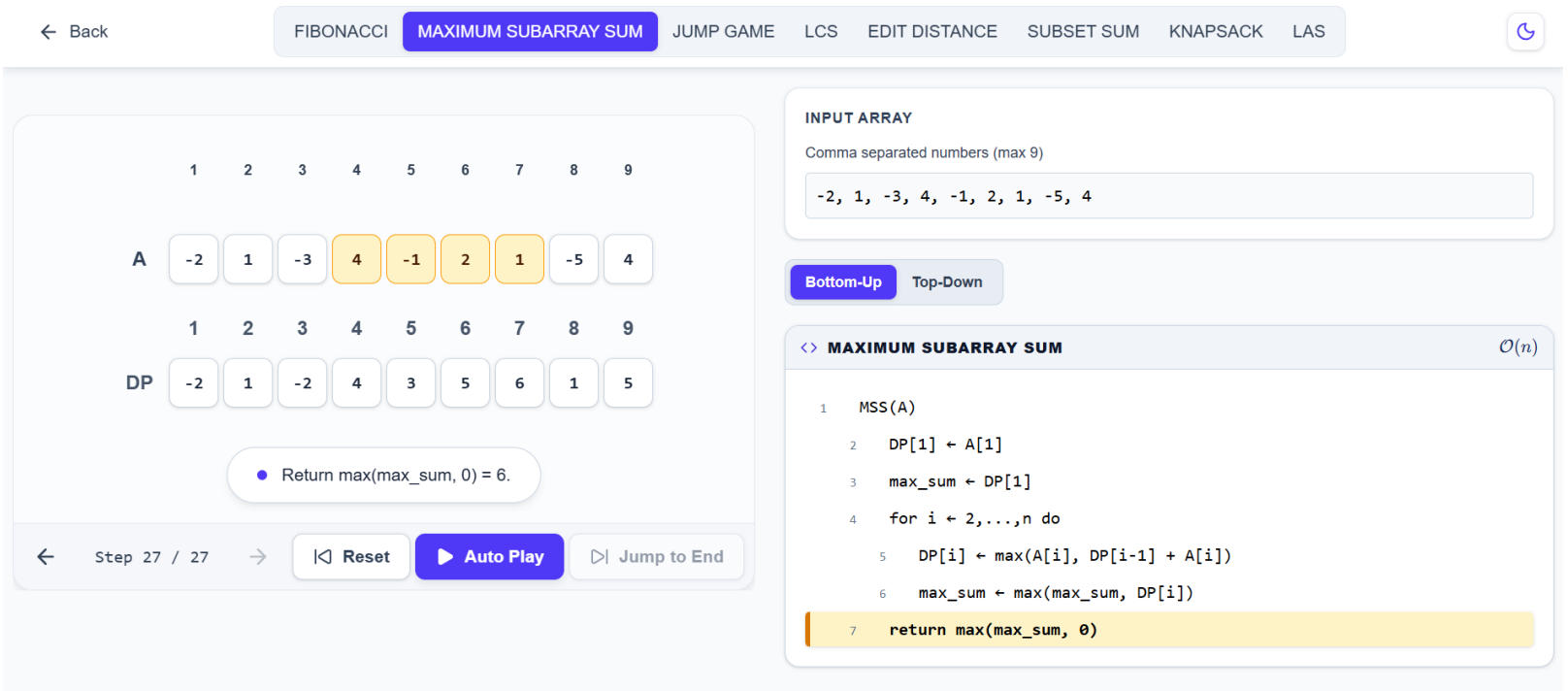
Base Case: $R[1] = A[1]$

Rekursion: $R[i] = \max\{ \underbrace{A[i]}_{\text{neu starten}}, \underbrace{R[i-1] + A[i]}_{\text{weiter anhängen}} \}$ für $2 \leq i \leq n$

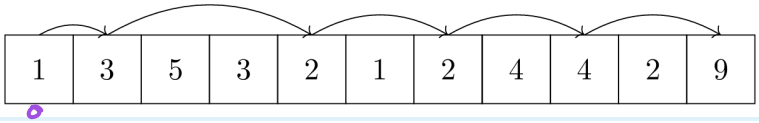
Berechnungsreihenfolge: for i = 2 ... n : berechne R[i]

Lösung: $\max\{ R[1], R[2], .., R[n], 0 \}$

Laufzeit: $O(n)$



Jump Game



Dimension: M[0.. n]

Teilproblem: M[k] := maximaler Index, der mit k Sprüngen erreichbar ist

Base Case: M[0] = 1 und M[1] = 1 + A[1]

2 Base Cases weil sich die Rekursion auf 2 vorherige Entries bezieht

Rekursion: $M[k] = \max \{ i + A[i] \mid M[k-2] < i \leq M[k-1] \}$ für $2 \leq k \leq n$

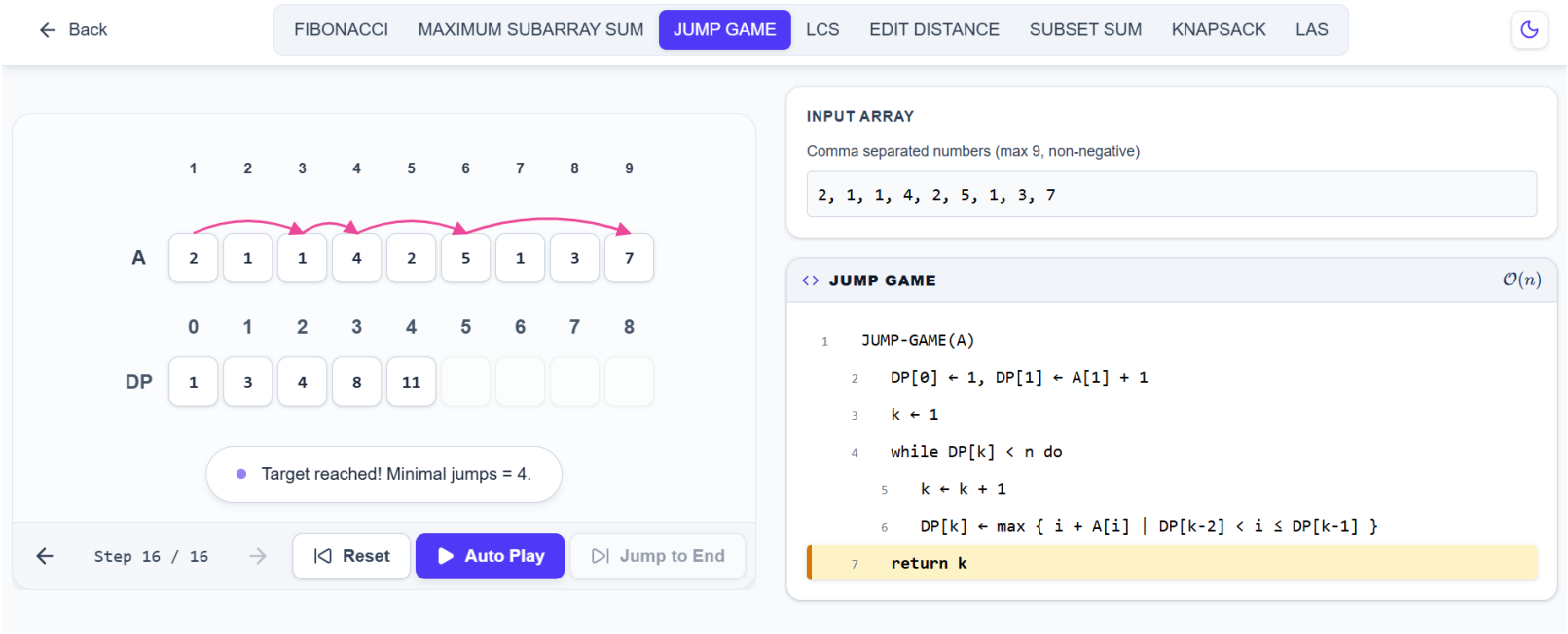
$i + A[i]$: k-ter Sprung

$M[k-2] < i \leq M[k-1]$: mögliche Indices, die wir mit k-1 Sprüngen erreichen können (ohne Sprünge zu verschwenden)

Berechnungsreihenfolge: for k = 2 ... n : berechne M[k]

Lösung: minimales k, sodass M[k] ≥ n

Laufzeit: O(n)



Längste Gemeinsame Teilfolge (LGT / LCS)

Dimension: $L[0..n][0..m]$

Teilproblem: $L[i,j]$:= Länge einer LGT von $(a_1, a_2, .., a_i)$ und $(b_1, b_2, ..., b_j)$

Base Case: $L[0,j] = 0$ für $0 \leq j \leq m$ (A ist leer)

$L[i,0] = 0$ für $0 \leq i \leq n$ (B ist leer)

Rekursion:

$$L[i,j] = \begin{cases} L[i-1, j-1] + 1 & \text{falls } a_i = b_j \\ \max\{L[i-1,j], L[i,j-1]\} & \text{falls } a_i \neq b_j \end{cases} \quad \text{für } 1 \leq i,j \leq n$$

entweder a_i oder b_j kommt nicht vor in der LGT

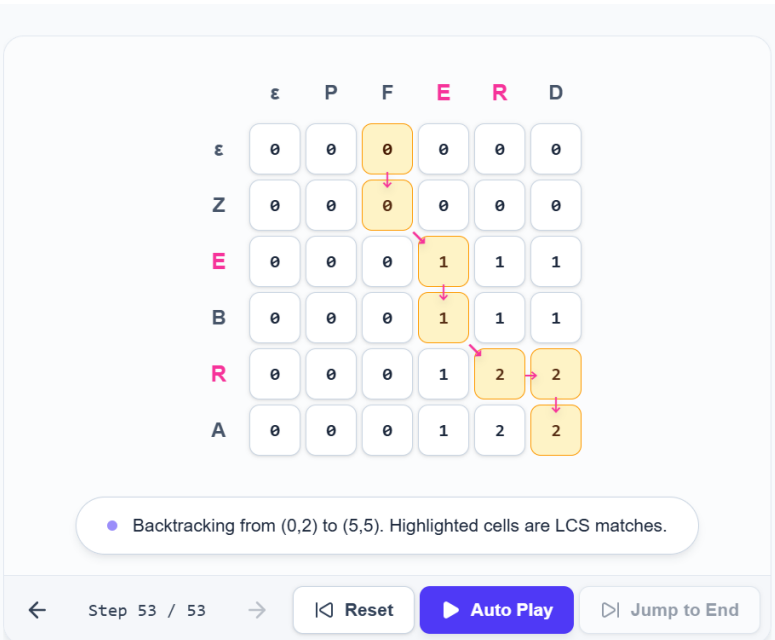
Berechnungsreihenfolge:

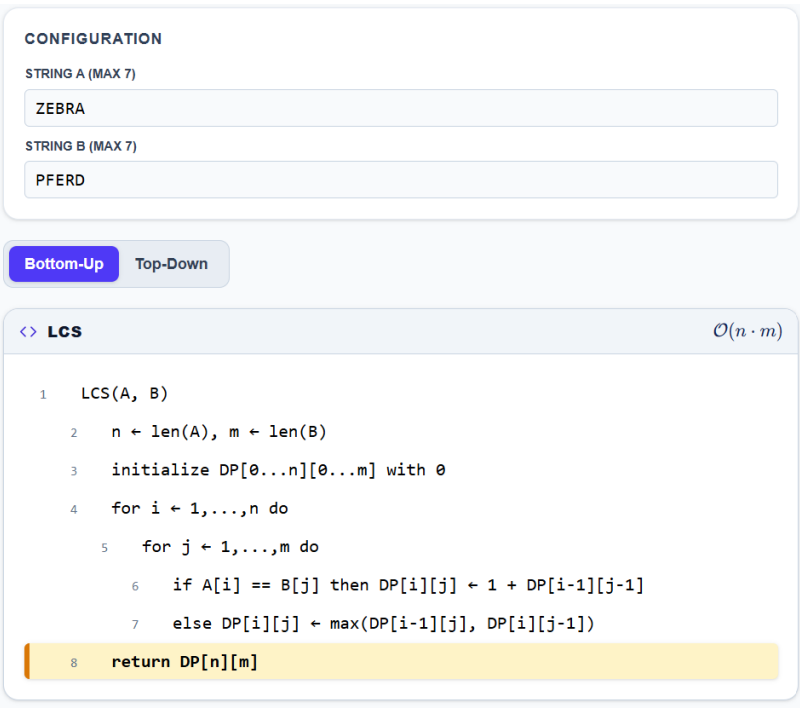
```
for i = 1...n
    for j = 1...m
        berechne L[i,j]
```

Lösung: $L[n,m]$

Laufzeit: $O(n \cdot m)$

(das Skript "versteckt" die Base Cases in der Rekursion)





Editierdistanz

- Einfügen: *Ein Zeichen an einer beliebigen Position in den String einfügen.*
- Löschen: *Ein Zeichen an einer beliebigen Position aus dem String löschen.*
- Ersetzen: *Ein Zeichen an einer beliebigen Position durch ein anderes Zeichen ersetzen.*

Dimension: $ED[0..n][0..m]$

Teilproblem: $ED[i,j]$:= Editierdistanz zwischen $(a_1, a_2, .., a_i)$ und $(b_1, b_2, .., b_j)$

Base Case:

$ED[0,j] = j$ für $0 \leq j \leq m$ (A ist leer)

$ED[i,0] = i$ für $0 \leq i \leq n$ (B ist leer)

Rekursion:

wir können das letzte Zeichen einfach ignorieren, weil es bei beiden gleich ist

$$ED[i,j] = \begin{cases} ED[i-1,j-1] & \text{falls } a_i = b_j \\ 1 + \min\{ED[i-1,j],\ ED[i,j-1],\ ED[i-1,j-1]\} & \text{falls } a_i \neq b_j \end{cases}$$

wir machen 1 Operation

$ED[i-1,j]$: a_i aus A löschen ($\Leftrightarrow$ a_i zu B hinzufügen)

$ED[i,j-1]$: b_j zu A hinzufügen ($\Leftrightarrow$ b_j aus B löschen)

$ED[i-1,j-1]$: a_i durch b_j ersetzen ($\Leftrightarrow$ b_j durch a_i ersetzen)

Berechnungsreihenfolge:

```
for i = 1...n
    for j = 1...m:
        berechne ED[i,j]
```

Lösung: $ED[n,m]$

Laufzeit: $O(n\ m)$

	ε	A	D	L	E	R
ε	0	1	2	3	4	5
E	1	1	2	3	3	4
U	2	2	2	3	4	4
L	3	3	3	2	3	4
E	4	4	4	3	2	3

Backtracking complete. Path highlighted.

Step 43 / 43 | Reset | Auto Play | Jump to End

EDIT DISTANCE $O(n \cdot m)$

```
1  EDIT-DISTANCE(A, B)
2    n ← len(A), m ← len(B)
3    initialize DP[0...n][0...m], fill base cases
4    for i ← 1,...,n do
5      for j ← 1,...,m do
6        if A[i] == B[j]
7          DP[i][j] ← DP[i-1][j-1]
8        else
9          DP[i][j] ← 1 + min(
9            DP[i-1][j], DP[i][j-1], DP[i-1][j-1])
11   return DP[n][m]
```

Programmier-Muster

Bottom-up:

1) DP-Array erstellen (im Pseudo code weggelassen)
2) BC eintragen
3) for-Schleife(n) anfangen
4) Rekursionsgleichung, direkt im DP table eintragen
5) ausserhalb der Schleife(n), die Lösung auslesen und returnen

```
FIBONACCI(n)
    F[0] ← 0
    F[1] ← 1
    for i ← 2,...,n do
        F[i] ← F[i-1] + F[i-2]
    return F[n]
```

```
MSS(A)
    DP[1] ← A[1]
    max_sum ← DP[1]
    for i ← 2,...,n do
        DP[i] ← max(A[i], DP[i-1] + A[i])
        max_sum ← max(max_sum, DP[i])   → macht das auslesen der Lösung einfacher
    return max(max_sum, 0)
```

```
LCS(A, B)
    n ← len(A), m ← len(B)
    initialize DP[0...n][0...m] with 0   ↳ this solves the base case implicitly
    for i ← 1,...,n do
        for j ← 1,...,m do
            if A[i] == B[j] then DP[i][j] ← 1 + DP[i-1][j-1]
            else DP[i][j] ← max(DP[i-1][j], DP[i][j-1])
    return DP[n][m]
```

Top-down:

1) DP-Array erstellen und alles mit -1 füllen
2) compute(..) aufrufen und Lösung auslesen
3) compute Funktion beginnen
4.) Falls es schon berechnet wurde, direkt returnen (= memoization)
5.) Base cases mit if's abfangen (Reihenfolge egal)
6.) Sonst Rekursion mit compute(..) Aufrufen ausdrücken und im DP table speichern
7) Den eben berechneten Wert returnen

```
FIBONACCI(n)
    initialize F[0...n] with -1
    return compute(n)
compute(i)
    if F[i] ≠ -1: return F[i]
    if i ≤ 1: F[i] ← i
    else F[i] ← compute(i-1) + compute(i-2)
    return F[i]
```

```
MSS(A)
    initialize DP[1...n] with -1
    compute(n)
    return max(max(DP), 0)
compute(i)
    if i == 1 return A[1]
    if DP[i] ≠ -1 return DP[i]
    DP[i] ← max(A[i], compute(i-1) + A[i])
    return DP[i]
```

```
LCS(A, B)
    initialize DP[0...n][0...m] with -1
    return compute(n, m)
compute(i, j)
    if i == 0 or j == 0 return 0
    if DP[i][j] ≠ -1 return DP[i][j]
    if A[i] == B[j]
        DP[i][j] ← 1 + compute(i-1, j-1)
    else DP[i][j] ← max(compute(i-1, j), compute(i, j-1))
    return DP[i][j]
```

Aufgaben:

Ein Einbrecher plant, einige Häuser in einer Strasse mit n Häusern auszurauben.
Haus i hat einen bestimmten Geldbetrag $\text{money}[i]$, der gestohlen werden kann, wobei $1 \leq i \leq n$.

Es gibt eine Sicherheitsregel: Er darf nicht zwei direkt benachbarte Häuser ausrauben, sonst geht der Alarm los.

Was ist der maximale Geldbetrag, den er stehlen kann, ohne den Alarm auszulösen?

Beispiel: $\text{money} = [2, 7, 9, 3, 1]$

- Option A: Haus 1 + Haus 3 + Haus 5 → Geldbetrag: $2 + 9 + 1 = 12$
- Option B: Haus 1 + Haus 4 → Geldbetrag: $2 + 3 = 5$

  ...
- → Der maximale Geldbetrag ist 12.

Dimension: DP[1 .. n]

Teilproblem: DP[i] := maximaler Geldbetrag, wenn wir nur die ersten i Häuser betrachten (und keinen Alarm auslösen)

Base Case: DP[1] = money[1] → wir brechen bei Haus 1 ein

DP[2] = max { money[1], money[2] } → wir brechen bei Haus 1 oder 2 ein, aber nicht beide (sonst Alarm)

Rekursion: DP[i] = max { DP[i-2] + money[i], DP[i-1] } für $3 \leq i \leq n$

- DP[i-2] + money[i]: bei Haus i einbrechen. Geld von ersten i-2 Häusern + Geld von Haus i
- DP[i-1]: Haus i überspringen: immer noch gleich viel Geld wie bei Haus i-1

Berechnungsreihenfolge: for i = 3 .. n: berechne DP[i]

Lösung: DP[n]

Laufzeit: O(n)

You are given an integer array cost where $\text{cost}[i]$ is the cost of i-th step on a staircase, where $0 \leq i \leq n-1$.

Once you pay the cost, you can either climb one or two steps.

You can either start from the step with index 0, or the step with index 1.

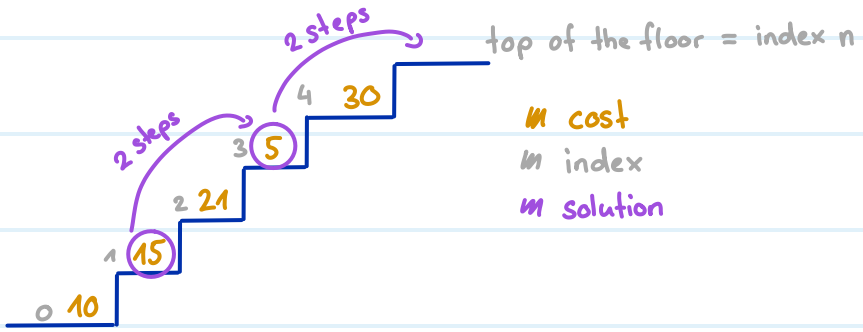
Return the minimum cost to reach the top of the floor.

Example: $\text{cost} = [10, 15, 21, 5, 30]$ (0 1 2 3 4)

Output: 20

Explanation: You will start at index 1.

- Pay 15 and climb two steps to reach index 3.
- Pay 5 and climb two steps to reach the top.
- The total cost is $15 + 5 = 20$.



Dimension: DP[0 .. n]

Teilproblem: DP[i] := minimum cost to reach step i

Base Case: DP[0] = 0 und DP[1] = 0 → wir können entweder bei Stufe 0 oder 1 beginnen

Rekursion: DP[i] = min { DP[i-2] + cost[i-2], DP[i-1] + cost[i-1] }

mit einem 2er Schritt zu i: Kosten bis i-2 + Kosten für den Schritt ab Stufe i-2 zu i

mit einem 1er Schritt zu i: Kosten bis i-1 + Kosten für den Schritt ab Stufe i-1 zu i

Berechnungsreihenfolge: for i = 2 . . n: berechne DP[i]

Lösung: DP[n]

Laufzeit: O(n)

Bottom-up:

```java
public static int minCostClimbingStairs(int[] cost) {

  int n = cost.length;
  int[] DP = new int[n+1];

  // Base Cases:
  DP[0] = 0;
  DP[1] = 0;

  // Rekursion:
  for (int i = 2; i<=n; i++) {
    DP[i] = Math.min(DP[i-2]+cost[i-2], DP[i-1]+cost[i-1]);
  }

  // Lösung:
  return DP[n];
}
```

Top-down:

```java
public static int minCostClimbingStairs(int[] cost) {
  // DP initialisieren:
  int n = cost.length;
  DP = new int[n+1];
  for (int i = 0; i<DP.length; i++) DP[i] = -1;

  // Lösung:
  return compute(n);
}

public static int compute(int i) {
  // Base case:
  if (i == 0 || i == 1) return 0;

  // Memoization: (check if already computed)
  if (DP[i] != -1) return DP[i];

  // Rekursion:
  DP[i] = Math.min(compute(i-2)+cost[i-2], compute(i-1)+cost[i-1]);
  return DP[i];
}
```

The next year has n days. You need to travel to ETH with SBB on some of these days. The boolean array $\text{travel}[1...n]$ contains a 1 if we need to travel on day i, and a 0 otherwise.

There are three types of tickets to buy: They are valid for 1, 7, or 30 days and their cost is given by C_1, C_7 and C_{30}.

Find the minimum cost if you want to have a valid ticket on every day you need to travel.

Example: $\text{travel} = [1, 0, 1, 1, 1, 0, 1, 1, 1, 0, 1]$ and $n = 11$ and $C_1 = 1, C_7 = 5, C_{30} = 10$

We buy a 1-day ticket on day 1, a 7-day ticket on day 3, and a 1-day ticket on day 11.

Thus the minimum total cost is $C_1 + C_7 + C_1 = 1 + 5 + 1 = 7$

Dimension: DP[0 .. n]

Teilproblem: DP[i] := minimum cost for having a valid ticket on travel days up to day i

Base Case: DP[0] = 0

Rekursion:

↳ wir brauchen kein Ticket => Kosten gleich wie am Tag vorher

$$DP[i] = \begin{cases} DP[i-1] & \text{if travel}[i] = 0 \\ \min\{ DP[i-1] + C_1,\ DP[i-7] + C_7,\ DP[i-30] + C_{30} \} & \text{if travel}[i] = 1 \end{cases}$$

($:= 0$ falls $i-7 < 0$; $:= 0$ falls $i-30 < 0$)

- $DP[i-1] + C_1$: Kosten für erste i-1 Tage + Kosten für das Ticket für Tag i
- $DP[i-7] + C_7$: Kosten für erste i-7 Tage + Kosten für das 7 Tage Ticket für Tage i-6, .., i-1, i
- $DP[i-30] + C_{30}$: Kosten für erste i-30 Tage + Kosten für das 30 Tage Ticket für Tage i-29, .., i-1, i

Berechnungsreihenfolge: for i = 1 . . n: berechne DP[i]

Lösung: DP[n]

Laufzeit: O(n)